Web Security: Implementing Role-Based Access Controls and CSRF Protection

Section 1: Learn

What is Role-Based Access Control (RBAC)?

RBAC restricts access to resources based on user roles. It's commonly used in enterprise systems to ensure only authorized individuals perform sensitive operations.

Why Use RBAC?

- **Enhanced Security**: Prevents unauthorized access.
- **Simplified Management**: Easier to manage permissions across user groups.
- **Compliance**: Supports regulatory standards like HIPAA, GDPR.

RBAC Key Concepts

- **Users**: Authenticated individuals.
- **Groups**: Collections of users (e.g., department, project teams).
- **Roles**: Defined responsibilities with access permissions (e.g., ADMIN, VIEWER).
- **Permissions**: Actions allowed (view, edit, delete, etc.).

Interesting Anecdote

RBAC is modeled after real-world job hierarchies in organizations—like managers having more access than interns—making digital security policies easier to align with business needs.

Section 2: Practice

RBAC Schema Example

```
Users (id, username, password)

Roles (id, name)

Permissions (id, resource, action)

UserRoles (user_id, role_id)

RolePermissions (role_id, permission_id)
```

Role Hierarchy in Spring Security

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .antMatchers("/admin/**").hasRole("ADMIN")
                .antMatchers("/manager/**").hasRole("MANAGER")
                .anyRequest().authenticated()
            .and()
            .formLogin().loginPage("/login").permitAll();
    }
}
```

CSRF (Cross-Site Request Forgery)

What is CSRF?

An attack that tricks authenticated users into submitting malicious requests to a web app using their valid session.

CSRF Protection in Forms

- **Generate Token**: Unique per session
- **Include in Form**: As a hidden input
- **Validate**: Server checks token before processing request

Enabling CSRF Protection in Spring

```
http.csrf().csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse
());
```

CSRF Best Practices

- **Use Framework Defaults**: Spring Security provides built-in CSRF protection.
- **SameSite Cookies**: Add SameSite=Lax or Strict to block cross-site cookies.
- **Token Expiry**: Expire tokens periodically.
- **Rotate Tokens**: Regenerate after login or sensitive operations.

Full Security Configuration Example

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
   @Override
```

```
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .antMatchers("/admin/**").hasRole("ADMIN")
                .antMatchers("/manager/**").hasRole("MANAGER")
                .anyRequest().authenticated()
            .and()
            .formLogin().loginPage("/login").permitAll()
            .and()

.csrf().csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse());
    }
}
```

Section 3: Know More

FAQs

Q1: What's the difference between groups and roles?

- **Answer:** Groups are collections of users; roles are sets of permissions. A group can be assigned a role.

Q2: Why is CSRF a major concern in web apps?

- **Answer:** Because it uses your authenticated session to perform unintended actions like changing passwords or making transactions.

Q3: Can we disable CSRF in Spring Boot?

- **Answer:** Yes, but only for stateless APIs or services where CSRF is not relevant. Never disable it in form-based apps.

Q4: How is CSRF token stored?

- **Answer:** Typically in cookies or as hidden fields in forms.

Q5: Can RBAC be combined with method-level security?

- **Answer:** Yes. Use annotations like `@PreAuthorize("hasRole('ADMIN')")` to secure methods in your service layer.

These strategies—RBAC and CSRF protection—form the backbone of modern secure web applications, particularly those built with Spring Boot.